HACKAPERTUS 2026 · TRACK 2A · FHGR: AI-POWERED JOB INTERVIEW COACH

PrepTalk

An interview coach in which Apertus talks and judges, and a rule engine decides

Team NLP-Lab · Götz-Henrik Wiegand (PhD student, University of St. Gallen) · Online

Code: github.com/ghwiegand/PrepTalk · Demo video

Summary

PrepTalk helps 14–16-year-olds practise a Swiss apprenticeship interview in German, French or Italian. It asks occupation-specific questions, follows up on vague answers, answers the candidate's questions as the company, and then switches to coaching with evidence from the conversation and a concrete next step. Swiss German answers are accepted; the coach replies in Standard German.

Apertus 1.5 8B phrases reactions and follow-ups, judges answers and writes feedback contents. **FrameX controls the interview**: explicit rules determine question order, reasks, follow-ups and the transition to feedback. Deterministic checks constrain model output, with fixed fallbacks for failed calls. This separates fluent wording from the decisions that make an interview useful and repeatable.

> **Reported evaluation.** The original evaluation used synthetic applicants, two independent model judges and a separate comparison with FHGR reference feedback. The final coach met the project's regression checks in all 28 scenario interviews and averaged about **3.1 model calls per candidate answer**. Against FHGR's reference feedback, levels were within one step in **100 % of 141 assessed criteria**; **87 %** of evidence excerpts were verbatim. These are development results, not the official benchmark or a study with real teenagers.

How PrepTalk addresses the challenge

Requirement	Implementation and evidence
Apertus 1.5 8B	`swiss-ai/Apertus-v1.5-8B`, inference only, temperature 0; vLLM serving.
Local deployment, < 32 GB VRAM	Docker GPU profile; bf16 weights need about 16 GB, plus serving overhead. Evaluation used one NVIDIA L4.
< 5 calls per answer	Guard, reaction and judgement, with extra calls for company answers and feedback; about 3.1 on average.
Adaptive interview	Situation-specific plans, profile-conditional questions, one reask and targeted follow-ups.
Constructive feedback	11 criteria, four levels, evidence, up to two improvement points and one practice task.
Multilingual and voice-ready	de/fr/it; dialect-aware checks; spoken feedback rendering and an experimental local macOS voice client.
Reproducible deliverables	Source, Docker stack, README, visual architecture and evaluation scripts.

Reading the evidence

The report follows the judging dimensions: **Performance 50 %** (Section 2), **Consistency 25 %** (Section 3) and **Innovation 25 %** (Section 4). The rules comparison, group breakdown and reference-feedback study are identified separately. Technical constraints and model use are described next; limitations, reproduction commands and disclosure close the report.

1 Architecture and use of Apertus

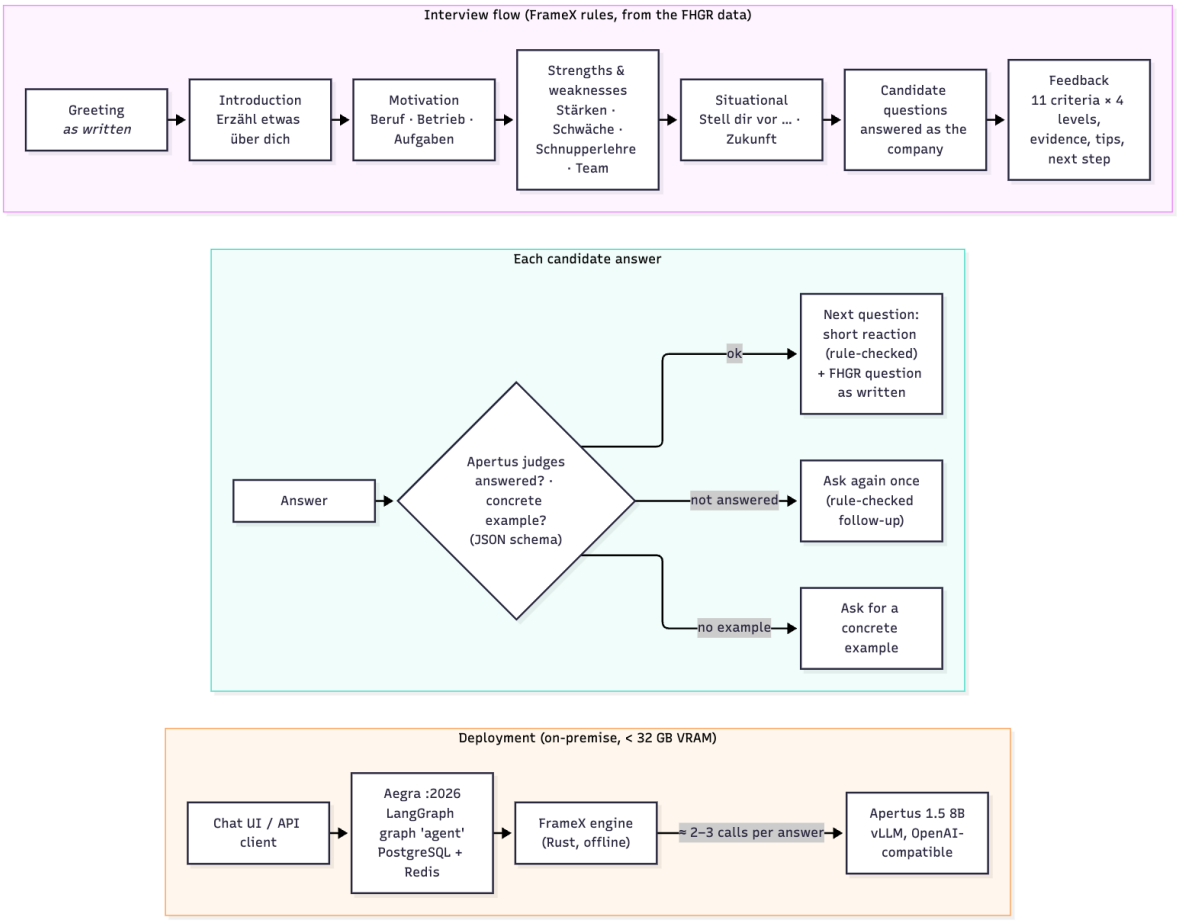


Figure 1. Interview flow, turn processing and deployment. The generated FrameX program selects the next action; Apertus supplies judgements and wording through an OpenAI-compatible endpoint.

Interview plan and state. FHGR's question bank supplies six stages: introduction, motivation, strengths and weaknesses, situational questions, candidate questions and feedback. Six situations have tailored plans of usually 8–12 main questions (phone calls: three), including occupation-specific and profile-conditional questions. Each turn replays recorded tool results without model calls, consumes the pending answer and continues until input is needed. JSON state lives in LangGraph checkpoints; shared FrameX globals require serialized turns within a process.

Model roles and fallbacks. Apertus reacts, judges, answers as the company and fills feedback contents. A normal answer costs a guard call, reaction and judgement with a follow-up check; pattern-matched guards avoid a call. `json_schema` falls back to `json_object` for rejected schemas. Failed calls use neutral wording, fixed questions or rule-based judgements. Company answers are checked against posting facts; Swiss training terms use a glossary.

Coaching and voice. Distress pauses the interview with support and referral; manipulation redirects to the pending question. A feedback request ends early. Checked feedback (Section 4) can be rendered for speech with `voice: true`. An experimental macOS client adds local Whisper base transcription via whisper.cpp and system-voice playback; it was not part of the reported evaluation.

User interface and demo. The terminal interface supports scenario, apprenticeship posting and language selection, followed by a guided interview with progress indicators. The accompanying demo video shows adaptive follow-ups, support for distress, resistance to manipulation, feedback with conversation evidence and an exported decision trace.

Serving and data. `LLM_BASE_URL` selects the OpenAI-compatible Apertus endpoint. The GPU profile uses Swiss AI's vLLM image, bf16 and a 16,384-token context. FHGR data is fetched at a pinned revision rather than redistributed. Evaluation candidates and companies are fictional; interactive state is stored in checkpoints.

2 Evaluation: performance

Performance 50 % The evaluation separates interview quality from hard regression checks. Independent judges, Claude Opus 5.5 and Gemini Pro, assess scenario criteria and five aspects: realism, adaptivity, feedback quality, age-appropriateness and language consistency. Hard checks count failures directly, including aborted interviews, wrong-language reactions, third-person feedback and guard-handled text quoted as evidence. All live coach calls, including retries, count toward the call budget.

Table 1. Rules comparison (28 interviews per configuration). Criteria scores without rules are pooled; with rules, development / new-scenario scores are shown separately.

Measure	Without rules	With rules
Interviews passing all hard checks	3 of 28	28 of 28
Scenario criteria met (Opus)	42 %	67 % / 54 %
Scenario criteria met (Gemini)	45 %	75 % / 67 %
Model calls per candidate answer	2.06	3.07



Figure 2. Both colours represent the Apertus 1.5 8B coach: orange shows Apertus 1.5 8B without rules; blue shows Apertus 1.5 8B with rules. Claude Opus 5.5 (left) and Gemini Pro (right) were used only as independent judges to rate the coach's responses on the displayed quality criteria; they did not generate the coach's responses. The hard-check pass rate is measured by regression checks. Scenario criteria met and hard-check pass rates are percentages; aspect ratings use a 0–100 display scale. Read each axis individually.

Largest gain: reliable execution. Only three of 28 interviews pass all hard checks without rules (11 %, rounded); all pass with rules. The average call count remains below five despite the added checks. Pooled scenario success rises from 42 to 60 % under Opus and from 45 to 71 % under Gemini, as shown in Figure 2. Development and new-scenario results are reported separately in Table 1; the lower new-scenario scores leave a generalisation gap.

Quality gains are uneven. Both judges rate the rule-based version higher on every plotted aspect. The largest visual change is in hard-check compliance; adaptivity and feedback quality remain weaker than language consistency and age-appropriateness. Deterministic checks address recurring failure modes, while a follow-up's relevance and the interpretation of a difficult answer still depend on Apertus.

Reading across experiments. This rules comparison is reported separately from the earlier per-group consistency breakdown (Section 3), the FHGR reference-feedback study and the evidence-driven pilot (Section 4). The group results below have not been recomputed from this comparison's aggregate scores; they describe the original evaluation runs.

3 Evaluation: consistency

Consistency 25 % The same flow templates, output checks and fallbacks apply across languages and profiles. The breakdown below describes the original 28 scenario runs, pooling development and held-out scenarios. It measures scenario criteria met, not a claim of equal performance across all users.

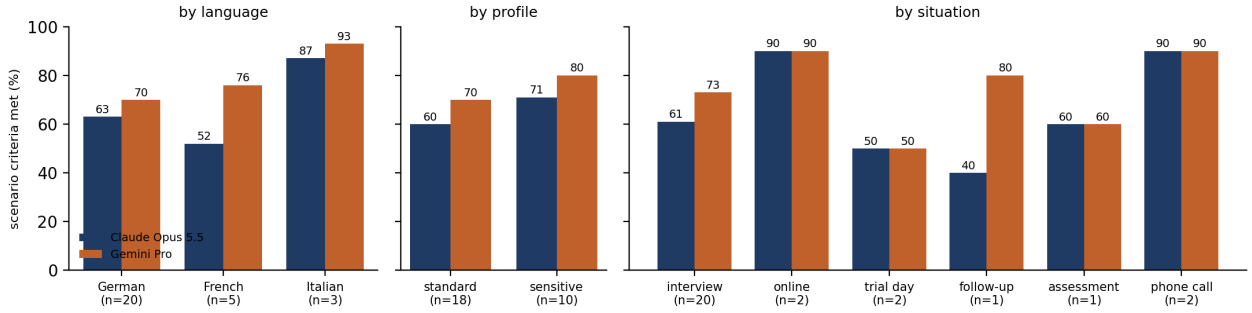


Figure 3. Original evaluation by language, profile and situation; *n* denotes scenario runs. Sensitive profiles include dyslexia, second-language speakers, interrupted schooling or apprenticeships, emotional moments, gender-atypical choices, strong self-doubt and dialect speakers.

Table 2. Original consistency breakdown: percentage of scenario criteria met, by judge. Groups are columns; parenthesized counts give the number of scenario runs.

	Language			Profile		Dialect		Difficulty		
Judge	German (20)	French (5)	Italian (3)	Sensitive (10)	Standard (18)	Dialect speaker (19)	Standard language (9)	Easy (5)	Hard (5)	Medium (18)
Opus	63 %	52 %	87 %	71 %	60 %	64 %	64 %	76 %	65 %	60 %
Gemini	70 %	76 %	93 %	80 %	70 %	71 %	80 %	80 %	81 %	70 %

	Situation					
Judge	Assessment (1)	Follow-up (1)	Interview (20)	Online (2)	Phone call (2)	Trial day (2)
Opus	60 %	40 %	61 %	90 %	90 %	50 %
Gemini	60 %	80 %	73 %	90 %	90 %	50 %

What is stable, and what varies. The original regression checks passed in all 28 interviews; calls per answer were similar across languages (de 3.04, fr 3.07, it 3.07). Sensitive profiles scored above standard profiles in this sample, while French scored lower than German or Italian under Opus. These small, differently composed groups do not establish fairness or a language effect. Per scenario run, Opus ranges from 0 to 100 % (mean 64, SD 26), Gemini Pro from 40 to 100 % (mean 74, SD 20). Several situation groups contain only one or two scenarios.

Independent checks of the rules. On 46 newly written guard cases, the two-stage guard handled distress 10/10 and manipulation 8/8, with no false alarm on 22 harmless cases; six mild self-doubt cases were tracked separately. Against FHGR expert annotations not used to tune the concreteness rule, precision/recall was de 0.95/0.79 (n = 326), fr 0.85/0.92 (29) and it 0.83/0.83 (21). Adding dialect markers raised dialect precision from 0.52 to 0.75. These local checks complement the model judges; they do not replace a user study.

4 Feedback and project contributions

Innovation 25 % FrameX is pre-existing MIT-licensed work from the Chair of Data Science and Natural Language Processing, University of St. Gallen. The project contributes its interview controller, compilation from FHGR data, answer checks, feedback assembly, trace interface and evaluation. Four small extensions were contributed to FrameX during the hackathon.

Feedback grounded in the conversation

The model returns levels, evidence, strengths, up to two improvement points with a stronger formulation, and one practice task as JSON. Rules produce the headings and required coaching blocks, reject evidence about nervousness or language proficiency, and rewrite or drop third-person text. A quotation is shown as verbatim only if it matches the candidate's words; unmatched evidence is labelled as a paraphrase. Phone-call feedback uses a deterministic checklist instead of the full rubric.

Table 3. Feedback against FHGR reference feedback: 141 assessed criteria in 13 transcripts.

Version	Exact	Within one	MAE	Verbatim
Criteria names only	49 %	95 %	0.56	69 %
+ Rubric level descriptions	65 %	100 %	0.35	0 %
+ Explicit evidence format	60 %	100 %	0.40	92 %
Final, vLLM schema enforced	**57 %**	**100 %**	**0.43**	**87 %**

The rubric improved calibration but initially lost evidence; an explicit evidence format restored it. The final signed bias was −0.14 levels (slightly harsher than the reference). Agreement within one level is a coarse measure: exact agreement was 57 %, and verbatim excerpts alone do not establish that a criterion or improvement is semantically justified.

Interview excerpt: S-05, Italian; synthetic applicant, shortened

APPLICANT *Mah, sicuramente in team! […] se c'è da fare un lavoro concentrato con i numeri, tipo inserire dati o fare l'inventario su Excel, mi piace anche mettermi […]*

COACH Quando hai lavorato da solo su Excel per fare l'inventario, cosa ti è successo e come l'hai gestito?

► A targeted follow-up asks for an experience, but presupposes that it happened: semantic assumptions remain a limitation even when the flow decision is correct.

What the hybrid design contributes

1. **Explicit control.** Apertus proposes yes/no/unclear verdicts; FrameX derives the next step. For concrete examples, model yes + rule vague becomes no, while model unclear + rule concrete becomes yes. This counteracts an early judge that accepted all 38 example checks, including stock phrases.
2. **Flows compiled from didactic data.** Questions, situations, occupational fields and profile conditions determine the program for a session, keeping FHGR's choices inspectable.
3. **Checked feedback and replay.** Templates constrain the feedback structure, while recorded tool results support replay without repeated model calls. The HTML decision trace shows questions, answers, judgements and rule decisions for review by developers or teachers.

An alternative we did not adopt

An evidence-driven pilot asks Apertus to extract a claim, situation, own contribution and result with quotes; FrameX then selects a follow-up for the missing component. Across two paired comparisons of 56 interviews each with hard applicants, Opus rated it 60 % versus 57 %, within judge variation, but unnecessary follow-ups doubled (70 versus 33). Extraction found situation and own contribution in only about 63 % of answers. The pilot remains off by default (`COACH_SEQUENCES=1` enables it): a higher judge score did not justify worse follow-ups.

5 Error analysis and limitations

Reading development transcripts and receipts exposed errors beyond aggregate judge scores:

Failure	Observed evidence	Response
Over-permissive judge	130/133 answers accepted; 38/38 example checks positive.	Deterministic checks on concreteness, company knowledge and concrete steps.
Needless repairs	23 repair calls, 19 in Italian after bare JSON strings.	Accept supported reply shapes; use fixed wording instead of repairing reactions.
Feedback prompt violations	Third person in 11/12 early feedbacks; mandatory hints missing.	Structured contents, rule assembly and targeted rewrite.
Robustness failures	Timeouts; feedback whitespace at the token limit.	Per-call fallbacks and recovery of usable truncated JSON.
Unused scenario hooks	Two of nine expected opening hooks missed.	Still open; encode relevant hooks explicitly.

Limits of the evidence. Gemini Flash simulates applicants; there has been no study with real teenagers. Model judges can miss errors and disagree, and development results exceed held-out results. Rules constrain selected failure modes; they do not guarantee grounded paraphrases, suitable advice or correct social judgements. Contradictions, arrogance and belittling others remain difficult for the 8B model. Small language and situation groups limit comparisons. Swiss German is understood but not generated. On the reported L4, median turns took about 12 s and feedback 60–80 s. Shared FrameX globals serialize turns within a process.

6 Reproducibility

From the repository root, configure `track_2a/.env` using its example, then run `make run` for Aegra on port 2026, PostgreSQL and Redis. Set `LLM_NAME`, `LLM_BASE_URL` and `LLM_API_KEY` for Apertus. `make run-gpu` also starts the GPU model; `make chat` opens the terminal client. `ARGS="--lang fr --trace trace.html"` selects a language and exports the trace. `make voice-setup` prepares the optional macOS voice client.

Evaluation modules under `track_2a/evaluation/` cover simulation, independent judges, regression checks, reference feedback and consistency. Gemini Flash simulates at temperature 0.9; the coach uses 0. Simulator and judge APIs are evaluation tools, separate from the deployed coach. Run local modules from `track_2a/` with `PYTHONPATH=src` and explicitly configured credentials; see the README. Docker pins FrameX and FHGR revisions; `uv.lock` pins Python dependencies.

The numbers are historical results, not fresh measurements of this checkout. Original run artifacts are not included, so the aggregates cannot be reconstructed from this repository alone. New experiments should retain code revision, endpoint configuration, transcripts and judge outputs.

Next steps. Improve follow-ups and scenario hooks, validate extraction before adopting the pilot, evaluate voice and latency, and study the coach with students and educators. Decision traces provide a starting point for a teacher review interface.

Team, disclosure and licence

Team NLP-Lab: Götz-Henrik Wiegand, PhD student, University of St. Gallen, submitting privately. FrameX is pre-existing university work, used from its public MIT-licensed repository. PrepTalk's application code was built during the hackathon. Development used AI coding assistance, including Claude Code.

Disclosure: Siegfried Handschuh, Chair of Data Science and Natural Language Processing at the University of St. Gallen, is a HackApertus jury member. His participation was announced in advance; he recuses himself from judging this submission.

Licence and references: code Apache 2.0; FrameX MIT; report and documentation CC BY 4.0. FHGR data retains its HackApertus-use licence and is fetched separately. Sources: FHGR / Profolio datasets, rubric and guidelines; Swiss AI Apertus 1.5; FrameX; vLLM, LangGraph and Aegra. Code: github.com/ghwiegand/PrepTalk.